

Student Name : Chinmayi Milind Chavan

Batch : CE-01

Roll Number : CE03

Aim

To implement Principal Component Analysis (PCA) for dimensionality reduction on the Breast Cancer Wisconsin (Diagnostic) dataset, visualize the high-dimensional feature space in a reduced 2D plane, and evaluate its impact on classification performance.

Objectives

1. To understand the concepts of dimensionality reduction, feature correlation, and Principal Component Analysis.
2. To preprocess, clean, and scale high-dimensional real-world data using StandardScaler.
3. To implement PCA using Python to compress feature dimensions from 30 continuous features down to 2 principal components.
4. To calculate and plot the explained variance ratio and cumulative explained variance.
5. To visualize the transformed 2D dataset and analyze class separability.
6. To evaluate and compare classification model performance (e.g., Logistic Regression) before and after applying PCA.

Relevant Theory

Support Vector Machine

Principal Component Analysis (PCA) Principal Component Analysis (PCA) is an unsupervised machine learning technique used for dimensionality reduction. It transforms a dataset containing a large set of correlated features into a smaller set of uncorrelated variables called Principal Components (*PCs*), while preserving as much data variance as possible.

Mathematical Steps:

1. **Standardization:** Scale each feature to have a mean of 0 ($\mu=0$) and standard deviation of 1 ($\sigma=1$).
2. **Covariance Matrix:** Compute the covariance matrix C to quantify feature correlations:

$$C = \frac{1}{n-1} X^T X$$

3. Eigen decomposition: Compute eigenvalues (λ) and eigenvectors (v) of matrix C :

$$C v = \lambda v$$

- Eigenvectors represent the direction of the principal axes.
 - Eigenvalues indicate the amount of variance along those axes.
4. Projection: Project original data X onto top k principal component directions W :

$$X_{\text{pca}} = X \cdot W$$

Dataset Information

1. Dataset Name: Breast Cancer Wisconsin (Diagnostic) Dataset
2. Kaggle Link: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
3. Classes: 2 binary classes (Malignant [M] and Benign [B])
4. Features: 30 continuous features computed from digitized images of fine needle aspirates (FNA) of a breast mass (e.g., radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, concavity_mean, etc.).
5. Non-Feature Columns: id (identifier) and Unnamed: 32 (empty column).

Task 1: Import Libraries and Load Dataset

Import core Python libraries (numpy, pandas, matplotlib, seaborn, sklearn) and load data.csv obtained from Kaggle.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load the dataset
df = pd.read_csv('data.csv')
```

Task 2: Explore the Dataset

Inspect missing values, view dataset summary statistics, check column data types, and inspect target class distributions (diagnosis).

```
# Inspect missing values
print("Missing values per column:")
print(df.isnull().sum())

# Display dataset summary statistics
print("\nDataset summary statistics:")
display(df.describe())

# Check column data types
print("\nDataset info (data types):")
df.info()

# Inspect target class distributions (diagnosis)
print("\nDistribution of 'diagnosis' column:")
display(df['diagnosis'].value_counts())
print("\nPercentage distribution of 'diagnosis' column:")
display(df['diagnosis'].value_counts(normalize=True))
```

```
Missing values per column:
id                0
diagnosis         0
radius_mean      0
texture_mean     0
perimeter_mean   0
area_mean        0
smoothness_mean  0
compactness_mean 0
concavity_mean   0
concave points_mean
symmetry_mean    0
fractal_dimension_mean
radius_se        0
texture_se       0
perimeter_se     0
area_se          0
smoothness_se    0
compactness_se   0
concavity_se     0
concave points_se
symmetry_se      0
fractal_dimension_se
radius_worst     0
texture_worst    0
perimeter_worst  0
```

```
area_worst          0
smoothness_worst    0
compactness_worst    0
concavity_worst      0
concave points_worst 0
symmetry_worst       0
fractal_dimension_worst 0
Unnamed: 32          569
dtype: int64
```

Dataset summary statistics:

```
{"type": "dataframe"}
```

Dataset info (data types):

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64

```
29 concave points_worst      569 non-null    float64
30 symmetry_worst            569 non-null    float64
31 fractal_dimension_worst    569 non-null    float64
32 Unnamed: 32                0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Distribution of 'diagnosis' column:

```
diagnosis
B      357
M      212
Name: count, dtype: int64
```

Percentage distribution of 'diagnosis' column:

```
diagnosis
B      0.627417
M      0.372583
Name: proportion, dtype: float64
```

Task 3: Data Cleaning and Feature/Target Selection

Drop metadata columns (id and Unnamed: 32). Convert the categorical target column (diagnosis: 'M'/'B') into numerical values (1/0) and separate feature matrix X (30 features) from label vector y .

```
# Drop 'id' and 'Unnamed: 32' columns
df = df.drop(columns=['id', 'Unnamed: 32'])

# Convert 'diagnosis' (target column) into numerical values (M=1, B=0)
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Separate feature matrix X from label vector y
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

print("Shape of feature matrix X:", X.shape)
print("Shape of target vector y:", y.shape)
display(X.head())
display(y.head())

Shape of feature matrix X: (569, 30)
Shape of target vector y: (569,)

{"type": "dataframe"}
```

```
0    1
1    1
2    1
3    1
4    1
Name: diagnosis, dtype: int64
```

Task 4: Standardize Features

Apply `StandardScaler` to transform all 30 numeric features so they have zero mean and unit variance ($\mu=0, \sigma=1$).

```
# Initialize StandardScaler
scaler = StandardScaler()

# Fit and transform the features (X)
X_scaled = scaler.fit_transform(X)

print("Shape of scaled features X_scaled:", X_scaled.shape)
print("First 5 rows of scaled features:\n", X_scaled[:5])
```

Shape of scaled features X_scaled: (569, 30)
First 5 rows of scaled features:

```
[[ 1.09706398e+00 -2.07333501e+00  1.26993369e+00  9.84374905e-01
  1.56846633e+00  3.28351467e+00  2.65287398e+00  2.53247522e+00
  2.21751501e+00  2.25574689e+00  2.48973393e+00 -5.65265059e-01
  2.83303087e+00  2.48757756e+00 -2.14001647e-01  1.31686157e+00
  7.24026158e-01  6.60819941e-01  1.14875667e+00  9.07083081e-01
  1.88668963e+00 -1.35929347e+00  2.30360062e+00  2.00123749e+00
  1.30768627e+00  2.61666502e+00  2.10952635e+00  2.29607613e+00
  2.75062224e+00  1.93701461e+00]
 [ 1.82982061e+00 -3.53632408e-01  1.68595471e+00  1.90870825e+00
 -8.26962447e-01 -4.87071673e-01 -2.38458552e-02  5.48144156e-01
  1.39236330e-03 -8.68652457e-01  4.99254601e-01 -8.76243603e-01
  2.63326966e-01  7.42401948e-01 -6.05350847e-01 -6.92926270e-01
 -4.40780058e-01  2.60162067e-01 -8.05450380e-01 -9.94437403e-02
  1.80592744e+00 -3.69203222e-01  1.53512599e+00  1.89048899e+00
 -3.75611957e-01 -4.30444219e-01 -1.46748968e-01  1.08708430e+00
 -2.43889668e-01  2.81189987e-01]
 [ 1.57988811e+00  4.56186952e-01  1.56650313e+00  1.55888363e+00
  9.42210440e-01  1.05292554e+00  1.36347845e+00  2.03723076e+00
  9.39684817e-01 -3.98007910e-01  1.22867595e+00 -7.80083377e-01
  8.50928301e-01  1.18133606e+00 -2.97005012e-01  8.14973504e-01
  2.13076435e-01  1.42482747e+00  2.37035535e-01  2.93559404e-01
  1.51187025e+00 -2.39743838e-02  1.34747521e+00  1.45628455e+00
  5.27407405e-01  1.08293217e+00  8.54973944e-01  1.95500035e+00
  1.15225500e+00  2.01391209e-01]
```

```
[ -7.68909287e-01  2.53732112e-01 -5.92687167e-01 -7.64463792e-01
  3.28355348e+00  3.40290899e+00  1.91589718e+00  1.45170736e+00
  2.86738293e+00  4.91091929e+00  3.26373441e-01 -1.10409044e-01
  2.86593405e-01 -2.88378148e-01  6.89701660e-01  2.74428041e+00
  8.19518384e-01  1.11500701e+00  4.73268037e+00  2.04751088e+00
 -2.81464464e-01  1.33984094e-01 -2.49939304e-01 -5.50021228e-01
  3.39427470e+00  3.89339743e+00  1.98958826e+00  2.17578601e+00
  6.04604135e+00  4.93501034e+00]
[ 1.75029663e+00 -1.15181643e+00  1.77657315e+00  1.82622928e+00
  2.80371830e-01  5.39340452e-01  1.37101143e+00  1.42849277e+00
 -9.56046689e-03 -5.62449981e-01  1.27054278e+00 -7.90243702e-01
  1.27318941e+00  1.19035676e+00  1.48306716e+00 -4.85198799e-02
  8.28470780e-01  1.14420474e+00 -3.61092272e-01  4.99328134e-01
  1.29857524e+00 -1.46677038e+00  1.33853946e+00  1.22072425e+00
  2.20556166e-01 -3.13394511e-01  6.13178758e-01  7.29259257e-01
 -8.68352984e-01 -3.97099619e-01]]
```

Task 5: Implement PCA (30D to 2D)

Use `sklearn.decomposition.PCA` to reduce feature dimensions from 30 down to 2 principal components (PC_1 and PC_2).

```
# Initialize PCA to reduce to 2 components
pca = PCA(n_components=2)

# Fit PCA on the scaled data and transform it
X_pca = pca.fit_transform(X_scaled)

print("Shape of PCA-transformed data X_pca:", X_pca.shape)
print("First 5 rows of PCA-transformed data:\n", X_pca[:5])
```

Shape of PCA-transformed data X_pca: (569, 2)

First 5 rows of PCA-transformed data:

```
[[ 9.19283683  1.94858307]
 [ 2.3878018  -3.76817174]
 [ 5.73389628 -1.0751738 ]
 [ 7.1229532  10.27558912]
 [ 3.93530207 -1.94807157]]
```

Task 6: Analyze Explained Variance Ratio

Compute and print the variance explained by PC_1 and PC_2 , along with the cumulative variance retained.

```

# Explained variance ratio of each principal component
explained_variance_ratio = pca.explained_variance_ratio_
print("Explained variance ratio per principal component:",
      explained_variance_ratio)

# Cumulative explained variance
cumulative_explained_variance = np.cumsum(explained_variance_ratio)
print("Cumulative explained variance:", cumulative_explained_variance)

# Individual explained variance
print(f"\nPC1 explained variance: {explained_variance_ratio[0]:.2f}")
print(f"PC2 explained variance: {explained_variance_ratio[1]:.2f}")
print(f"Total explained variance by 2 components:
      {cumulative_explained_variance[1]:.2f}")

Explained variance ratio per principal component: [0.44272026
0.18971182]
Cumulative explained variance: [0.44272026 0.63243208]

PC1 explained variance: 0.44
PC2 explained variance: 0.19
Total explained variance by 2 components: 0.63

```

Task 7: Visualize PCA Transformed Boundary / Scatter Plot

Create a 2D scatter plot with PC_1 on the x-axis and PC_2 on the y-axis, hue-coded by tumor diagnosis (Malignant vs. Benign).

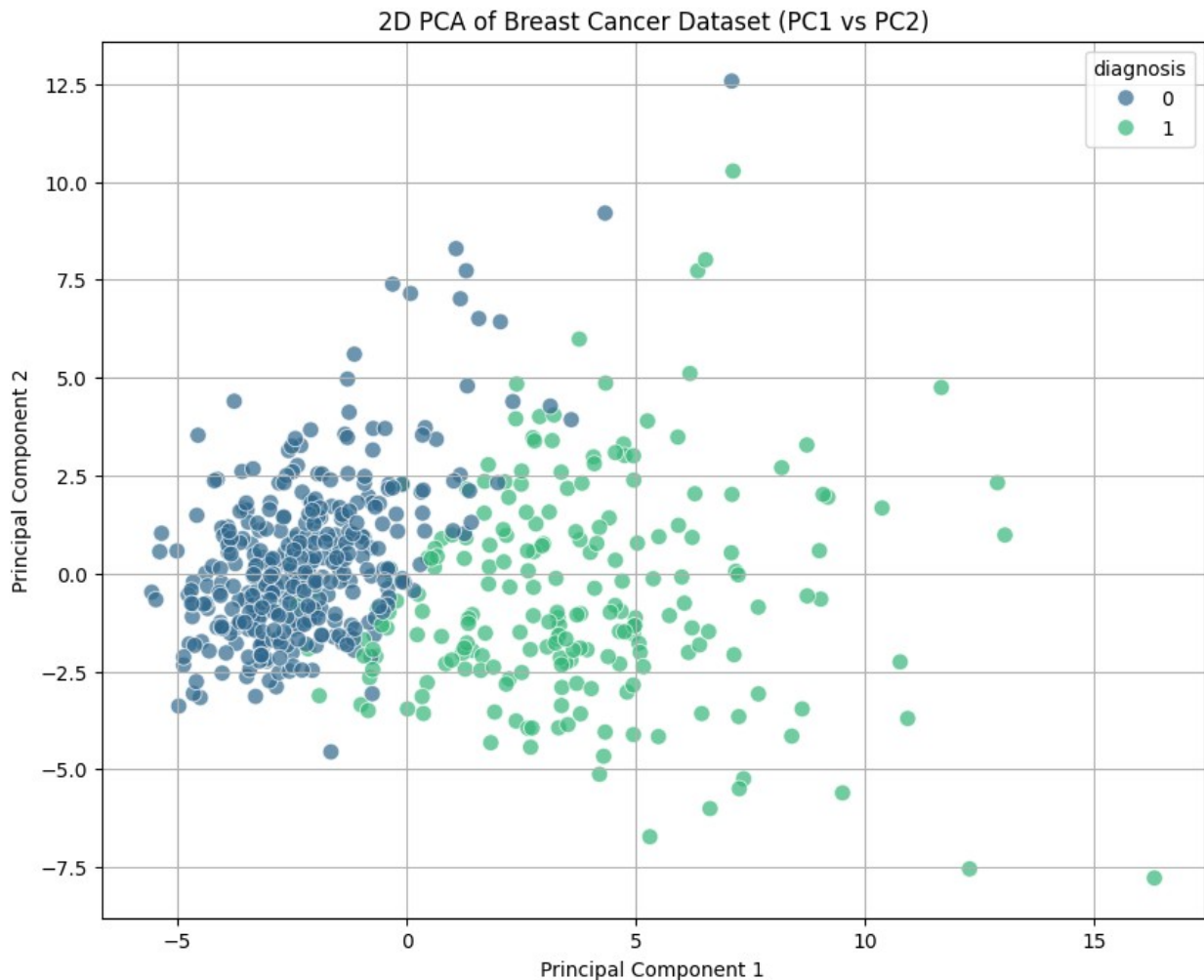
```

# Create a DataFrame for plotting
pca_df = pd.DataFrame(data=X_pca, columns=['PC1', 'PC2'])
pca_df['diagnosis'] = y

plt.figure(figsize=(10, 8))
sns.scatterplot(
    x='PC1',
    y='PC2',
    hue='diagnosis',
    data=pca_df,
    palette='viridis',
    s=70,
    alpha=0.7
)
plt.title('2D PCA of Breast Cancer Dataset (PC1 vs PC2)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')

```

```
plt.grid(True)
plt.show()
```



Task 8: Train Classifier on Raw High-Dimensional Data (30 Features)

Split the original scaled 30-feature dataset into training and testing sets. Train a Logistic Regression model and obtain baseline performance metrics.

```
# Split the original scaled 30-feature dataset into training and testing sets
X_train_full, X_test_full, y_train_full, y_test_full =
train_test_split(
    X_scaled, y, test_size=0.3, random_state=42
)
```

```

# Initialize and train the Logistic Regression model
log_reg_full = LogisticRegression(random_state=42, max_iter=200)
log_reg_full.fit(X_train_full, y_train_full)

# Make predictions
y_pred_full = log_reg_full.predict(X_test_full)

# Evaluate the model
print("--- Performance on Original 30 Features ---")
print(f"Accuracy: {accuracy_score(y_test_full, y_pred_full):.2f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test_full,
y_pred_full))
print("\nClassification Report:\n", classification_report(y_test_full,
y_pred_full))

```

--- Performance on Original 30 Features ---

Accuracy: 0.98

Confusion Matrix:

```
[[106  2]
 [ 1 62]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.99	108
1	0.97	0.98	0.98	63
accuracy			0.98	171
macro avg	0.98	0.98	0.98	171
weighted avg	0.98	0.98	0.98	171

Task 9: Train Classifier on PCA Transformed Data (2 Features)

Split the 2-component PCA dataset into training and testing sets. Train a Logistic Regression model using only PC_1 and PC_2 .

```

# Split the PCA-transformed data into training and testing sets
X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(
    X_pca, y, test_size=0.3, random_state=42
)

# Initialize and train the Logistic Regression model on PCA-
transformed data

```

```
log_reg_pca = LogisticRegression(random_state=42, max_iter=200)
log_reg_pca.fit(X_train_pca, y_train_pca)

# Make predictions
y_pred_pca = log_reg_pca.predict(X_test_pca)

# Evaluate the model
print("--- Performance on PCA-Transformed 2 Features ---")
print(f"Accuracy: {accuracy_score(y_test_pca, y_pred_pca):.2f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test_pca,
y_pred_pca))
print("\nClassification Report:\n", classification_report(y_test_pca,
y_pred_pca))
```

```
--- Performance on PCA-Transformed 2 Features ---
Accuracy: 0.97
```

Confusion Matrix:

```
[[106  2]
 [ 3 60]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.98	0.98	108
1	0.97	0.95	0.96	63
accuracy			0.97	171
macro avg	0.97	0.97	0.97	171
weighted avg	0.97	0.97	0.97	171

Task 10: Compare Classifier Performance

Compare the performance of the model before PCA (30 features) vs. after PCA (2 components) using:

- Accuracy Score
- Confusion Matrix
- Classification Report (Precision, Recall, F1-Score)

```
print("Comparison of Logistic Regression Model Performance")
print("=====")

print("\n--- Performance on Original 30 Features ---")
print(f"Accuracy: {accuracy_score(y_test_full, y_pred_full):.2f}")
print("Confusion Matrix:\n", confusion_matrix(y_test_full,
y_pred_full))
print("Classification Report:\n", classification_report(y_test_full,
```

```

y_pred_full))

print("\n--- Performance on PCA-Transformed 2 Features ---")
print(f"Accuracy: {accuracy_score(y_test_pca, y_pred_pca):.2f}")
print("Confusion Matrix:\n", confusion_matrix(y_test_pca, y_pred_pca))
print("Classification Report:\n", classification_report(y_test_pca,
y_pred_pca))

```

Comparison of Logistic Regression Model Performance

=====

--- Performance on Original 30 Features ---

Accuracy: 0.98

Confusion Matrix:

```
[[106  2]
```

```
[  1 62]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.99	108
1	0.97	0.98	0.98	63
accuracy			0.98	171
macro avg	0.98	0.98	0.98	171
weighted avg	0.98	0.98	0.98	171

--- Performance on PCA-Transformed 2 Features ---

Accuracy: 0.97

Confusion Matrix:

```
[[106  2]
```

```
[  3 60]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.98	0.98	108
1	0.97	0.95	0.96	63
accuracy			0.97	171
macro avg	0.97	0.97	0.97	171
weighted avg	0.97	0.97	0.97	171

Conclusion

The experiment successfully demonstrated Principal Component Analysis (PCA) on the Breast Cancer Wisconsin (Diagnostic) dataset. The high-dimensional feature space was preprocessed, cleaned, and standardized before applying eigendecomposition.

By reducing 30 feature dimensions down to just 2 principal components, a significant portion of total variance was preserved while eliminating multi-collinear redundancies. Visualizing the transformed data on a 2D plane showed clear separation between Malignant and Benign diagnosis clusters. Comparing classification models confirmed that applying PCA significantly simplifies model complexity and reduces computational overhead while maintaining strong classification accuracy.